

Abstract

Mustache is a logic-less templating language used widely in web development. Its multi-language design enforces a strict separation between templates and program logic. Template engines should not compute anything, only substitute values and iterate over data. This makes templates safe and easy to reason about, but it creates a problem. Any logic the template needs, whether an item is first in a list, whether a description is needed, or whether something is empty, has to be precomputed or specified after checking the output. As a result, this means someone has to manually add this in, which defeats the point of automation.

This dissertation aims to utilise standard ML's properties to preprocess the JSON before it is parsed through into the mustache template so that the final rendered output has 'logic'. The preprocessor sits in front of any standard Mustache implementation and requires no changes to the template layer.

The implementation uses CM-Lex and CM-Yacc to produce a abstract syntax tree (AST), with a lazy stream lexer feeding the parser. A renderer supports the core Mustache feature set, and the preprocessor handles null fields and array position flags. SML's type system provides compile-time verification that the required transformations are complete before rendering.

The prototype is tested with 47 unit tests across three suites covering the parser, preprocessor, and renderer, all passing on SML/NJ (Standard Meta Language/ New Jersey) version 110.99.9. Known limitations include parsed but unrendered support for partials and delimiter changes, hard-coded build paths, and a restricted variety of test inputs. The contribution is a working parser and preprocessor that demonstrates a potential in making templating safer and having 'logic'.

Acknowledgements

You may wish to include an acknowledgement. This should be a maximum of one page and is usually much less. Thank whoever may have helped you in any way, both serious and a bit of fun.

The acknowledgements are written in the first person singular, i.e. “I would like to thank my ...”

Contents

Abstract	i
Acknowledgements	i
List of Figures	iv
1 Introduction	1
1.1 Functional Languages Usage in Biomedical Engineering	4
2 Literature Review	5
2.1 Template theory and Systems Design	5
2.2 Templating Engines Across Platforms	7
2.3 Alternative Templating Systems	8
2.4 Functional Languages for Template Processing	10
2.5 Takeaway	12
3 Requirements and Analysis	13
3.1 Problem Statement	13
3.2 Functional Requirements	13
3.3 Non-Functional Requirements	14
3.4 Problem Decomposition	14
3.4.1 Parsing the Mustache Template	14
3.4.2 Enriching the JSON Data	15
3.4.3 Rendering the Template	15
3.4.4 CLI and Build System	16
3.4.5 Testing	16
3.5 Known Gaps and Limitations	16
3.6 Evaluation Strategy	17
3.7 Legal and Ethical Considerations	18
4 Design	19
4.1 Choice of Implementation Language	19
4.2 Architectural Choice: Preprocessor Rather Than New Engine	21

4.3	Parser Design	21
4.3.1	Parser Generators	22
4.3.2	AST Shape	22
4.4	Preprocessor Design	23
4.4.1	Operating on the JSON Datatype	23
4.4.2	Public Interface	23
4.4.3	A Worked Example	24
4.5	Renderer Design	24
4.5.1	Tree-Walking Interpreter	24
4.5.2	Truthiness	25
4.5.3	Dot-Separated Path Resolution	25
4.6	Build and Deployment	25
5	Implementation and Testing	27
5.1	Implementation Overview	27
5.2	Implementing the Parser	27
5.3	Implementing the Preprocessor	28
5.4	Implementing the Renderer	29
5.5	Coding Traps Encountered	29
5.6	Testing Strategy	30
5.7	Test Results	30
5.8	Limitations Confirmed by Testing	31
6	Results and Discussion	32
6.1	Summary of Findings	32
6.2	Goals Achieved	32
6.3	Comparative Analysis	33
6.4	Discussion	34
6.5	Further Work	35
7	Conclusions	37
A	Appendix Title	39

List of Figures

1.1	A snippet template (left) and the email it produces when filled in with one customer's data (right). The double braces mark the slots where information is inserted from a dataset	1
2.1	Model-View-Controller State and Message Sending (adapted from Krasner & Pope, 1988)	6
4.1	The preprocessing pipeline. The Mustache template is parsed into an abstract syntax tree (AST); the JSON data is parsed into a raw JSON value; the pre-processor walks the raw JSON and emits an enriched JSON value containing the contextual flags the template requires; the renderer consumes both the AST and the enriched JSON to produce the final output string.	19

1. Introduction

Templating engines are one of the most common operations in modern software. Commonly used in web development, template engines are systems that generate outputs such as emails or documents from data files like JSON (JavaScript Object Notation). Any tool designed to generate source code from a specification will, at some point, separate a fixed structural skeleton from the variable data used to populate it. This separation is what a template system provides, enabling mass automation of tailored outputs. A practical example of this is a company that automates SDK (Software Development Kit) generation for its clients, since each client is different. Another example can be shown in 1.1, the left shows what the template would look like for the developer, and the right shows the generated output, which gets delivered to the customer, 'Sarah'.

Template	Rendered email
Dear {{name}},	<i>Dear Sarah,</i>
Thank you for your order.	<i>Thank you for your order. Your copy of</i>
Your copy of {{book_title}}	<i>Wuthering Heights will be delivered on</i>
will be delivered on	<i>14 May.</i>
{{delivery_date}}.	<i>Best wishes,</i>
Best wishes,	<i>The Bookshop</i>
The Bookshop	

Figure 1.1: A snippet template (left) and the email it produces when filled in with one customer's data (right). The double braces mark the slots where information is inserted from a dataset

Templating is embedded in our everyday lives: from web pages, emails, documents, and code generation. While famously used for web development, templating engines have expanded far beyond web development to DevOps and CI/CD pipelines [1], infrastructure as code [2], network automation [3], and general software tasks [4] and more.

Despite the significance of templating engines, there is a fair security issue that exists with them. Server-side template injections (SSTI) is a vulnerability which arises when user-supplied input is embedded into a template without being treated as protected data, allowing the attacker to inject executable code into the template engine [5]. While many developers confuse these flaws for cross-site scripting (XSS), which is an attack on the browser, exploiting vulnerabilities in client-side code, SSTI is more dangerous [6]. Being a risk highlighted by Open Worldwide Application Security Project (OWASP), SSTI is an attack on the server,

meaning the attacker may gain access to the server's file system and environment [7], [8]. The canonical study of the problem by Kettle [9] demonstrates that this typically can escalate to remote code execution (RCE), granting attackers full control over servers or applications. The real-world impact of SSTI is highlighted by the high-profile breach of uber in 2016, in which Orange Tsai exploited Flask's Jinja2 engine on 'rider.uber.com' by simply setting a profile name to a mathematical expression within a template tag, specifically `''7'*7'`, which the Uber mail server evaluated and returned as `'7777777'` in the confirmation email, earning him a \$10,000 bug bounty for what was ultimately full remote code execution [10]. SSTI is not confined to one engine: documented exploits exist for Jinja2, Twig, Smarty, FreeMarker, Velocity, Thymeleaf, and Jade, among others, with many of the underlying engines downloaded millions of times a month [5], [11]. OWASP has listed SSTI vulnerabilities are among the most critical risks to the web applications for over a decade [7].

An overview of the security vulnerabilities of templating engines is discussed. The systematic analysis of 34 popular template engines across 8 programming languages found that 31/34 allowed RCE, while fewer than a third implement any built-in security protection, such as sandboxing or restricted execution [5]. Even where defences exist, they are routinely bypassed: a 2023 study of 7 widely-used PHP template engines identified 135 new template-escape bugs and synthesised working RCE exploits against 55 of them [12]. The vulnerability is also not a rarity in practice: it accounted for a measurable share of critical-risk findings in a commercial web application audit [5]. Remediation tends to be slow, extending the window during which affected applications remain exposed.

In response to these security issues is to take away the template's ability to execute code or involve business logic. Thus, enforcing a strict separation between presentation and application logic was argued for [13] on grounds of maintainability and security. A need for a template that cannot compute or be tricked into computing something the developer did not intend. Mustache released by Chris Wanstrath in 2009, introduces a 'logic-less' framework to separate business logic from presentation through reducing the available operations to variable substitution, section iteration and no conditionals or loops [14]. Because the syntax is so restricted, the same Mustache template can be rendered by an independent implementations in dozens of programming languages, including Ruby, JavaScript, Java, PHP, Haskell, and OCaml [15], [16]. Specifically, the price of this safety is what the literature calls the *preprocessing burden*. Anything that the template would have computed itself in a logic-friendly system, the developer must now compute in advance and embed in the data. If a list needs commas between items but not after the last one, the developer has to walk the list manually and attach an 'isLast' flag to each element. If a field might be missing, the developer has to add a companion 'has_field' flag to make the template's conditional render without missing it. None of this is hard, but all of it is tedious, error-prone, and untyped. Nothing checks that the developer remembered to add every flag, and a missing flag is silent

at compile time and only visible by inspecting the rendered output.

Logic-less templates are 'dumb' enough to evade these security issues; however, they have argued to have a trade-off. Logic-friendly templates are convenient but expose applications to security attacks, whereas logic-less templates are safe but divide business logic from presentation logic of the template. Critics [17], [18] have made a similar point of 'the logic has to go somewhere' [19]: instead of a logic-friendly template engine, the developer has to now precompute the 'logic' and embed the data into the template. The cost of preprocessing has two components, and the relationship between them matters more than either in isolation. The runtime cost is small. Research on type-based security in templating frameworks illustrates how dramatically architecture determines runtime cost: Samuel et al. [20] report that compile-time, type-based approaches to context-sensitive sanitisation incur overheads of 3-10%, against 78% to over 500% for purely runtime approaches measuring the same property. Preprocessing JSON to add contextual flags is a similarly bounded, compile-time-friendly operation. The cost that actually persists, however, is the developer's. Every new template, every change to the underlying data model, and every newly-added field potentially introduces a fresh set of flags that must be identified, computed, and embedded into the rendering context by hand. A list that gains a new sorting requirement needs a new flag; a field that becomes optional needs a companion 'has_field' flag added wherever the template references it; a section that becomes nested needs position flags propagated through the structure. Nothing in the toolchain checks that this work has been done: a missing flag produces a stray comma or a malformed sentence in the rendered output, with no error message and no compile-time warning. The preprocessing burden of logic-less templating is therefore not primarily a performance problem but a correctness and verification problem that recurs across the lifetime of the application, and it is this cost that the present work addresses.

This dissertation investigates whether Standard ML, a functional language, can automate that preprocessing while preserving Mustache's logic-less safety benefits. Functional programming properties, such as algebraic data types for tree-shaped data, exhaustive pattern matching, and robust static typing, are as effective for preprocessing automation as they are for traditional parsing and transformation tasks. The contribution of this project is a working prototype. It consists of a Mustache parser, a JSON preprocessor, and a renderer that consumes the enriched data. The preprocessor sits in front of any standard Mustache implementation, requires no changes to the Mustache specification, and uses Standard ML's pattern-matching exhaustiveness check to ensure that every transformation is applied consistently across nested data structures. All 47 unit tests across the three components pass, and the tool runs end-to-end from the command line.

Chapter 2 reviews the literature on template systems, examining alternative engines across platforms and investigating the parsing theory that underpins their implementation. Chapter 3 defines the preprocessor requirements, breaks the problem into manageable components,

and establishes the evaluation criteria. Chapter 4 justifies the major design decisions: language choice, architectural approach, and implementation strategy, against the alternatives considered. Chapter 5 describes the implementation, documents the technical challenges encountered, and presents the testing methodology and results. Chapter 6 evaluates the prototype against its original aims, compares the typed preprocessing approach to manual alternatives, and identifies directions for further work.

1.1 Functional Languages Usage in Biomedical Engineering

Functional languages have a long history in scientific computing, and BioCaml [21] is the most visible example of their adoption in biology: a comprehensive OCaml library for sequence analysis, file format parsing, and high-performance genomics. The case for typed functional languages in bioinformatics has been argued in detail by Berenger *et al.* [22], who identify the same properties this dissertation relies on. Strong static typing and Hindley-Milner inference allow complex hierarchical data structures to be represented precisely and checked for consistency at compile time, while algebraic data types and pattern matching let programs follow the shape of the data, and the compiler's exhaustiveness check catches missing cases before a long-running analysis crashes on input it was not prepared for. The preprocessor described in this dissertation uses an algebraic data type to represent the parsed Mustache template and exhaustive pattern matching to guarantee that every contextual flag a template requires has a corresponding case in the preprocessing function: the same engineering values, applied to a different domain.